

UE 7 : Mobility and Smart Cities

Philippe Canalda

Philippe.Canalda@univ-fcomte.fr

Part 1, 23/09/2025, a.m. was :

- 1h15 of presentation of Louage problem : definition, specification and solving.
- 1h of course introducing orders and combinatorial algorithm by enumeration : greedy algorithm, enumeration with or without orders, enumeration with cut and price (when the solution computed will not improve the best results already computed), a recursive algorithm also known as dynamic programming algorithm with or without cut and price.
- 1h of mini-project-presentation
- The remaining part was devoted to practical work

Part 2 , 23/09/2025 p.m., is :

- 1h of presentation of multimodal itinerary generation problem : definition, specification and solving.
- 1h of course introducing orders (partial, total), graph properties among symmetry, transitivity.
- 1h of practical work to pursue exact solving by enumeration and price and cut and share + transitive closure
- [1h of MSC project (Ph Canalda part 2)]

Cimo: an efficient 2-phases calculator of multimodal itineraries for real trans-territories based on a dynamic programming

- To be downloaded separately



Cimo: an efficient 2-phases calculator of multimodal itineraries for real trans-territories based on a dynamic programming

Authors:

❖ **Idriss HASSINE**
idriss.hassine.2015@ieee.org

❖ **Philippe CANALDA**
philippe.canalda@femto-st.fr

Subject of practical work n ° 1&2 (or homework)

- n Prg1 = return the 'making change' greedy approach, taken from the course.
The initialization of the data is carried out "hard", that is to say by using `int L [7] = {5, 2, 1, 0.5, 0.2, 0.1, 0.05}`
The preferred language is JAVA. It could be C, or C ++, Python, ...
 - » As support for your work, you must provide, even in a draft version: the class diagram, the factory fab (int) which updates the instantiation of the test, the compilation command, the command or the script execution (s), the results files, an analysis of these results and their reviews.
- n Prg2 = exhaustive calculation of all solutions without recursivity.
The calculation is long. Mostly printing to the screen or to a file.
The solutions are displayed on the screen with an overload of the `toString (...)` function. The result can be redirected to a `solutions_n ° test_output.txt` file

Subject of practical work n ° 1&2 (or homework)

- Work to be done in practical work continued: part giving change continued
 - n Prg3 = exhaustive calculation by recursion (also called dynamic programming algorithm) where all the solutions are generated in an in-depth walk. The solution of the current calculation is stored in an array of the same length as L and which contains the number of units of the available values. The combinations are displayed on the screen as they happen, when a solution is valid. This approach makes it possible to avoid the consumption of allocated memory (also known as RAM). However, I / O are special files called buffers which consume resources (slowness).

Suite TP1 - change part continued

- n Continuation Prog 3: The data displayed on the screen will be redirected to a file AllTheSolutionsTest1.txt When this file is validated you suffix it with CORRECT_ You can restart the java execution Prg1> AllTheSolutionsTest1.txt You will compare the 2 files with the command : diff CORRECT_AllSolutionsTest1.txt AllSolutionsTest1.txt
- n Prg3 bis: from prg 3 write prg3bis which reverses the order of the generation of solutions by favoring the maximum number of units of the value being processed.
- n Prg4 = You calculate all the solutions that you do not display, as you go. If you store the solutions in a 2D array of solutions, in the order in which the valid combinations are found, then a subsequent display following that order can be compared with the contents of the CORRECT_ ... file and will show no difference.

Continuation of suiteTP1

- n Prg5 = modify program 3 so as to evaluate the cost of a solution according to the number of units of the values. Calculate the best solution according to this cost by displaying successive values that improve. Display the trace of this execution in a file `CORRECT_SolutionsWhichImproveIncrementally.txt`
Calculate the number of solutions displayed, and the display economy, using the `wc -l` command. Calculate with the `time`, or `date` command, the execution time programs 3 and 5.
- n Prg6 = modify program 5 or program 4 so that the best calculated solution is stored in a solution table. Evaluate the computation time saved and the display savings.
- n Prg7 = from program 6, implement the cut. To do this, the solution cost value being calculated must be calculated as you go. When the “partial” cost is greater than the `best_cost`, you stop recursive processing. This amounts to adding an `end` (if then else additional to the recursive processing).

Continuation of suiteTP1

- n program 7 bis: How effective is the cut, and/or the gain, when the order implemented in prg3bis is implemented in prg 7?
- n Program 8 that you could do but that I don't ask: is equivalent to calculating a sorted subset of the 5 best solutions.
- n Program 8' which you could do but which I don't ask: is equivalent to calculating the best solution as well as the 2 less worst solutions. The first of the less worse solutions will have a better cost but for an amount to be made positive. The second less worse will have a better cost but for an amount to be made negative. Would you be able to realize this program !!!!!

— Relationship part

- n The Boolean function is Transitive (int Relation (int [] [])) which evaluates whether a relation is transitive or not. The relation passed in parameter is the most simplistic with t [i] [j] taking the values -1 if no relation, 0 on the diagonal, and a positive value of cost to characterize the trnsnision of the node i towards the node j.

Continuation of MSC project

— Project activity

- 2 functions to be developed here f1 and f2. the incomplete displacement graph can be obtained by interrogating a GIS. The matrix is somewhat sparse because the paths obtained by transitivity are not initialized => a saving of time, access to the GIS and therefore a saving of money when the number of calls is counted. An example of Graph g not complete thus :

$$g = \begin{pmatrix} 0 & 1 & 8 & -1 \\ -1 & 0 & 5 & 2 \\ 1 & -1 & 0 & -1 \\ 3 & 2 & -1 & 0 \end{pmatrix}$$

f1 consists in making the transitive closure. We traverse all the arcs of g: $t[i][j]$. If $t[i][j] > 1$ I go looking for all the $t[j][k] > 0$. For such a couple $t[i][j] + t[j][k]$ if it is $> t[i][k]$ existing, I replace this $t[i][k]$.

Part2

Continuation of MSC project

— f1 pursued

- It is Dijkstra's algorithm that calculates the shortest paths from any point to any point.

Warning ! There is a degree of difficulty between isTransitive and this Transitive closure. During a complete walk over the matrix roll, if a modification occurs, and therefore an addition of edge is issued from the transitivity computation, it is necessary to redo a complete walk, in search of an improvement of the transitive closure. We stop when there has been no improvement or modification during the complete walk over the matrix roll.

Continuation of MSC project

— f2 :

- The f2 function uses a list of people positions. One then determines the point within the list of all the (discrete) positions of the problem by calculating the total value of all the displacements of the people to converge towards this point.

We keep the point that minimizes the overall movements of the people of the group of students using the application.

In order to, then, be able to generate the routes of each one, it is sufficient to modify the time matrix into a matrix of cells. If the cell is null, it matches -1. Otherwise, the cell will be made up of a cost in time, or in km, or a more complex table depending on the mode (bike, foot, car, etc.) and the time of day for taking congestion into account.

Continuation of MSC project

— f2 pursued:

- ... This cell is also made up of the route in text format which explains how to go from i to j. Going from i to k through j will correspond to the concatenation of the route t [i] [j] with that t [j] [k]. And that's all folk!

You have the innovative elements that many mobility solutions have not yet implemented;)

unrolling

- Apply the previous program on a short but realistic example ...
 - A size $|L|=2$,
 - An ordered decreasing list, or not ordered
 - A list that shows the interest to generate exhaustively (all the) solutions
 - A solution to which you may insert optimization improvement (like the price and cut, if any)

unrolling

- Apply the previous program on a short but realistic example ...
 - A size $|L|=2$,
 - An ordered decreasing list, or not ordered
 - A list that shows the interest to generate exhaustively (all the) solutions
 - A solution to which you may insert optimization improvement (like the price and cut, if any)
- $L=\{3, 2\}$, $m= \dots$

unrolling

- Apply the previous program on a short but realistic example ...
 - A size $|L|=2$,
 - An ordered decreasing list, or not ordered
 - A list that shows the interest to generate exhaustively (all the) solutions
 - A solution to which you may insert optimization improvement (like the price and cut, if any)
- $L=\{3, 2\}$, $m=7$, ...

« The eight queens »

- Given, a size of the chessboard and a number of queens to place
- Calculate the combinations of queen positions
 - All
 - Some according to heuristics (positions of the rider)
 - Discussion

Chapter basics

- Ordered data structures
- Trellis
- Boolean algebra
- Applications
- Exercises

Chapter basics : ordered Data Structures

- Notions
- Algebra of Boole

Chapter basics : ordered DS

- Relations, binary relations, some properties
 - A relation designates whatever subset of cartesian product of 2 or more variables
- Example 1
 - $\text{Prod_cart} = A \times B$ avec $A = \{a,b,c\}$ and $B = \{e1,e2,e3,e4\}$
then $\text{Prod_cart} = \{(a,e1), (a,e2), (a,e3), (a,e4), (b,e1), (b,e2), (b,e3), (b,e4), (c,e1), (c,e2), (c,e3), (c,e4)\}$
accepts the subset $R = \{(a,e2), (b,e1), (c,e4)\}$

Chapter basics : ordered DS

- Binary relation
 - A binary relation over E is a subset of $E \times E$
- Example 2
 - $\text{Prod_cart } E \times E$, avec $E = \{x, y, z\}$
Matrix representation or array-based shaping

(x,x)	(x,y)	(x,z)
(y,x)	(y,y)	(y,z)
(z,x)	(z,y)	(z,z)

Cartesian
Product

R1

(x,x)		(x,z)
	(y,y)	
(z,x)		

R2

(x,x)	(x,y)	
		(y,z)
(z,x)		(z,z)

Ch1, special relations

- Diagonal relation
 - $\Delta = \{(x,x), (y,y), (z,z)\}$
- Reflexive relation
 - Includes the diagonal
- Symmetrical relation
 - Foreach (x,y) in $E \times E$ then (y,x) in $E \times E$
 - R1 is symmetrical but not reflexive
- Anti-symmetrical relation
 - Foreach x et y , (y,x) not in R if (x,y) in R , excepted for $x=y$
 - R2 is anti-symmetrical but is not reflexive

R1

(x,x)		(x,z)
	(y,y)	
(z,x)		

R2

(x,x)	(x,y)	
		(y,z)
(z,x)		(z,z)

Ch1, special relations

- Transitive relation
 - If (x,y) and (y,z) in R then (x,z) in R
- The opposites : irreflexive, asymmetrical and intransitive
 - R is irreflexive if it does not contains any item from the diagonal
 - It is asymmetrical if it is not symmetrical for any couple of R
 - It is intransitive if it is not transitive for any couple of R

R3

(x,x)	(x,y)	(x,z)
		(y,z)
		(z,z)

R4

X,x	X,y		X,t
Y,x	Y,y		Y,t
		Z,z	
T,x	T,y		T,t

Example of solving a relation property problem

- Compose the algorithm which allows verifying a relation R is transitive

Example of solving a relation property problem

- Compose the algorithm which allows verifying a relation R is transitive
 - » // Representation of data : the 2-dimension array of the relation and the boolean result variable is_transitive
 - » `int[3][3] R3={{1,1,1},{0,0,1},{0,0,1}};`
`Boolean is_transitive = true;`
 - » `for (int i=0; ((i<3) && (is_transitive));i++) {`
 `for (int j=0; ((j<3) && (is_transitive));j++) {`
 // look for a good (x,y)
 `if R3[i,j] {`
 // look for (y,z) in the y-lineonly
 `for (int k=0; ((k<3) && (is_transitive));k++) {`
 `if R3[j,k] {`
 // verify if R3[x,k] exists
 `is_transitive = is_transitive && R3[i,k];`
 `}`
 `else ; // the current item is not valid => we treat the next item if it matches`
 `}`
 `} else ; // no item => wed treat the next one if any`
 `}`
} }

Example of solving a relation property problem (idea of exam)

- What happens to the previous algorithm if we want to determine whether a relation is intransitive or non-transitive?
 - Exploit what we already know is feasible !

2nd example of solving a relation property problem

- Modify the algorithm which allows verifying a relation R is transitive ...
 - » // Representation of data : the 2-dimension array of the relation and the boolean result variable is_transitive
 - » int[3][3] R3={{1,1,1},{0,0,1},{0,0,1}};
Boolean is_transitive = true;
 - » for (int i=0; ((i<3) && (is_transitive));i++) {
 for (int j=0; ((j<3) && (is_transitive));j++) {
 // look for a good (x,y)
 if R3[i,j] {
 // look for (y,z) in the y-lineonly
 for (int k=0; ((k<3) && (is_transitive));k++) {
 if R3[j,k] {
 // verify if R3[x,k] exists
 is_transitive = is_transitive && R3[i,k];
 }
 else ; // the current item is not valid => we treat the next item if it matches
 }
 } else ; // no item => wed treat the next one if any
 }
}

Ch1, special relations

- Qualify R3 et R4 ?

- R3

- Not reflexive, anti-symmetrical and transitive

- R4

- Reflexive, symmetrical and transitive

R3

(x,x)	(x,y)	(x,z)
		(y,z)
		(z,z)

R4

X,x	X,y		X,t
Y,x	Y,y		Y,t
		Z,z	
T,x	T,y		T,t

- Personal practical work :
 - [The 3 queens on a chessboard 3x3] optional
 - The algorithm wich implements intransitivity
 - Continue the mini-project

Ch1, special relations

- Qualify R5 and R6 ?

- R5

- irreflexive, asymmetrical and intransitive

- R6

- It is nothing ...

or rather it is non-reflexive, non-symmetrical and non-transitive

R5

	(x,y)	
		(y,z)
(z,x)		

R6

(x,x)	(x,y)	(x,z)
		(y,z)
(z,x)		